Inventory System — UE5.7 Blueprints

Documentation v0.7

Folder Structure

```
Content/
└── Inventory/
    ├── AC_InventoryComponent — core logic
    ├── BPI_Interactable — world interaction interface
    ├── BPL_InventoryUtils — function library (logging)
    ├── E_LogSeverity — logging severity enum
    ├── Input/
    │   ├── IA_Interact — Enhanced Input action (trigger: Started)
    │   ├── IA_Inventory — Enhanced Input action (trigger: Started)
    │   └── IMC_Inventory — Input Mapping Context (I = inventory, E = interact)
    ├── Data/
    │   ├── S_ItemData — item definition struct
    │   ├── S_InventorySlot — runtime slot struct
    │   └── DT_Items — item database (uses S_ItemData)
    ├── UI/
    │   ├── WBP_Inventory — main inventory panel
    │   ├── WBP_InventorySlot — single slot tile
```

```
│   ├── WBP_DragDrop — drag visual
│   ├── WBP_LootPanel — two inventories side by side ✓
│   └── BP_DragDropOperation — drag drop operation data carrier
└── Items/
    ├── BPI_Item — item behaviour interface
    ├── BP_Pickup_Base — parent pickup actor
    ├── BP_Pickup_Sword — example child
    └── BP_Lootable_Base — container actor with own AC_InventoryComponent ✓
```

Data Layer

Enum — ItemType

`Weapon`, `Armor`, `Tool`, `Material`, `Usable`

Drives item behaviour — `Usable` → `OnUse`, `Weapon`/`Armor` → `OnEquip`, `Material` → no direct action.

Enum — E_LogSeverity

Used to Log error paths through code and print out errors with its type.

Value	**Color**	**Notes**
Info	Green	General information
Warning	Yellow	Non-critical e.g. inventory full
Error	Red	Developer mistake e.g. invalid ItemKey
Severe Error	Red	Fatal — prints then Quit Game

S_ItemData (Struct)

Static item definition. Lives in `DT_Items`, never changes at runtime.

Field	**Type**	**Notes**
Name	String	Display name
Weight	Integer	Per unit weight
Type	ItemType	Enum
Texture	Texture2D	Icon
Stackable	Boolean	Can stack?
MaxStack	Integer	Max per slot

S_InventorySlot (Struct)

Runtime slot state. Minimal by design.

Field	Type	Notes
ItemKey	Name	DT row name — stable ID
Quantity	Integer	Current amount

Empty slot = `ItemKey None`, `Quantity 0`.

DT_Items (Data Table)

One row per item type. Row names are item IDs — set intentionally e.g. `Sword_Iron`, `Potion_Health`.

Interfaces

BPI_Interactable

Function	**Inputs**
Interact	Interactor (Actor)

BPI_Item

Function	**Inputs**
OnPickup	Interactor (Actor)
OnUse	Interactor (Actor)
OnDrop	Interactor (Actor)

BPL_InventoryUtils

LogInventoryError

Input	**Type**
Message	String
Caller	Object
Severity	E_LogSeverity

Format: `[Inventory] ActorName : Severity Message`. Prints to screen and log. Severe Error calls Quit Game.

AC_InventoryComponent ✓

Reusable Actor Component. Drop onto any character for full inventory, input, and interact functionality. Single player only.

Variables

Variable	Type	Notes
Slots	S_InventorySlot[]	Source of truth — always SlotMax length
SlotMax	Integer	Set per actor in Details panel
SlotsUsed	Integer	Refreshed from Slots.Length at start of each operation
Inventory_Weight	Integer	Updated via UpdateWeight
CurrentInteractable	Actor	Set/cleared by pickup actors
InventoryWidget	WBP_Inventory	Cached widget reference
PauseOnOpen	Boolean	Pause game when inventory opens

Event Dispatchers

`OnInventoryUpdated` — fires after any successful state change.

Input

On Begin Play: adds `IMC_Inventory`, binds `IA_Interact` (Started) and `IA_Inventory` (Started).

Use Started trigger on `IA_Inventory` to prevent multiple toggles from one key press.

`GetLocalPlayerSubsystem` result is guarded with **IsValid** before `Add Mapping Context` — prevents "Accessed None" error on PIE stop during level teardown.

Begin Play

Calls `FillSlotsArrayToMax` — loops until `SlotMax` empty slots exist in the Slots array.

Functions

SetInteractable(Actor) / ClearInteractable Called by pickup actors on overlap begin/end.

OpenInventory

- Creates `WBP_Inventory`
- `Sets UIOpen to true`
- Calls `SetComponentReference(self)` on widget after creation
- Sets input mode to Game and UI / Game Only
- Shows/hides cursor
- Pauses/unpauses if `PauseOnOpen` true

CloseUI

- Sets UIOpen to false
- Set input to game only&Show cursor to false
- checks whether Loot widget is open and closes either Lootpanel or Inventory

FillSlotsArrayToMax Runs on Begin Play. Fills Slots array with empty `S_InventorySlot` entries (None key, 0 quantity) up to `SlotMax`. Ensures fixed-length array for safe index access.

FindSlotByKey Searches Slots array for matching ItemKey.

Output	**Type**	**Notes**
SlotFound	Boolean	False if not found
ItemKey	Name	
ExistingQuantity	Integer	
ArrayIndex	Integer	-1 if not found

FindEmptySlot Loops Slots array, returns index of first slot where `ItemKey == None`. Returns -1 if inventory is full.

UpdateWeight Full recalculate from Slots array. Always call before firing dispatcher.

AddItem ✓ | Input | Type | Notes | |---|---|---| | Name | Name | ItemKey | | ItemQuantity | Integer | | | AutoFill | Boolean | False = one attempt, used by loot panel |

Returns: `-1` invalid item, `0` success, `>0` remainder.

Refactored flow:

1. Refresh `SlotsUsed` from `Slots.Length`
2. Check DT row exists — return -1 if not
3. `FindSlotByKey` — item exists in inventory?
4. If yes and `ExistingQuantity < MaxStack`:
    - All items fit → `Set Array Elem` with combined quantity, return 0
    - Overflow → top off existing slot to MaxStack, remainder continues
5. `FindEmptySlot` for new slot placement
    - `ItemQuantity <= MaxStack` → `Set Array Elem` at empty index, return 0
    - `ItemQuantity > MaxStack` → fill slot to MaxStack, `FindEmptySlot` for remainder
    - No empty slot → log Warning "Inventory is full", return remainder
6. `UpdateWeight` → fire dispatcher

Uses `Set Array Elem` throughout — never `ADD`. Slots array stays fixed at `SlotMax` length.

RemoveItem ✓ | Input | Type | |---|---| | Item Name | Name | | Amount | Integer |

- `ExistingAmount - Amount > 0` → `Set Array Elem` with reduced quantity
- `ExistingAmount - Amount <= 0` → `Set Array Elem` with None key and 0 (clears slot without shifting)
- Not found → `LogInventoryError` Warning

Uses `Set Array Elem` instead of `Remove Index` to preserve fixed array length and slot indices. Always calls `UpdateWeight` and fires `OnInventoryUpdated`.

MoveItem ✓ Direct array manipulation for drag and drop. Does not use AddItem/RemoveItem.

Input	**Type**
SourceComponent	AC_InventoryComponent
SourceIndex	Integer
TargetComponent	AC_InventoryComponent
TargetIndex	Integer
DraggedQuantity	Integer

Logic:

1. Get source and target slot data
2. Same item?
    - Yes → full merge possible → `Set Array Elem` combined quantity, clear source
    - Yes → overflow → top off target to MaxStack, set source to remainder
    - No → target empty → move source to target, clear source
    - No → swap source and target slot data
3. Fire `OnInventoryUpdated` on **both** components — required for cross-inventory moves so both panels refresh

---

WBP_Inventory ✓

Variables

Variable	**Type**
Inventory Ref	AC_InventoryComponent
SlotWidgets	WBP_InventorySlot[]

Layout: Canvas Panel → Border → Uniform Grid Panel (10 per row), Exit Button (bottom centre).

Functions

SetComponentReference(AC_InventoryComponent) Full widget initialisation. Called after widget creation by any system that owns this widget (component's `OpenInventory`, `WBP_LootPanel`'s `SetupLootPanel`).

1. SET `InventoryRef` = input component
2. `CreateSlots`
3. Call `RefreshInventory` via `OnInventoryUpdated` dispatcher — deferred through dispatcher so slot widgets are fully ready before `InitSlot` runs

CreateSlots Called from `SetComponentReference`. For Loop 0 to `SlotMax - 1` (Last Index calculated once before loop). Uses Construct to create each `WBP_InventorySlot` passing Index and self. Stores in `SlotWidgets` array, adds to Uniform Grid at `Column = Index % 10`, `Row = Index / 10`.

Reads `SlotMax` from `InventoryRef` — requires valid ref.

RefreshInventory Loops `SlotWidgets` array by index. Index < filled count → `InitSlot` with slot data. Empty slot → `InitSlot` with None key and 0.

Event Construct

Binds `OnInventoryUpdated` dispatcher to `RefreshInventory` using Create Event node referencing the Refresh custom event. Binding lives in Construct (EventGraph) — cannot be done inside a function in UE5 Blueprints.

WBP_InventorySlot ✓

Variables (set via Construct at creation):

- `Index (Integer)` — position in grid and Slots array
- `InventoryRef (AC_InventoryComponent)` — for reading slot data
- `ShiftDown (Boolean)` — tracked via OnKeyDown/OnKeyUp
- `CtrlDown (Boolean)` — tracked via OnKeyDown/OnKeyUp

Set `bIsFocusable` to true in widget settings. Call Set Keyboard Focus in `OnMouseButtonDown`.

Layout: Canvas Panel (95x100), dark semi-transparent background, icon Image, quantity Text Block bound to Quantity variable.

Functions

InitSlot(Key, Quantity)

- Sets Quantity variable
- DT lookup → Set Brush from Texture on icon
- Shows icon and quantity text if Quantity > 1
- Row Not Found → hides both (empty slot state)

Events

OnMouseButtonDown GET slot from `InventoryRef.Slots` at Index. If `ItemKey != None` → Detect Drag If Pressed → return result. If empty → return default Event Reply.

OnDragDetected GET slot data from `InventoryRef` at Index. DT lookup for texture → pass to `WBP_DragDrop`. Calculate drag amount:

- Ctrl held → 1
- Shift held → `Truncate(Quantity / 2)`
- Neither → full Quantity

Create `WBP_DragDrop` with item texture. Create `BP_DragDropOperation` with ItemKey, Amount, InventoryRef, Index. Return operation.

OnDrop Cast Operation to `BP_DragDropOperation`. Cast success → `MoveItem(SourceInv, SourceIndex, InventoryRef, Index, Amount)` → return true. Cast failed → return false.

WBP_LootPanel ✓

Two `WBP_Inventory` instances placed side by side. Each marked as **Is Variable** in the designer hierarchy — required so `SetupLootPanel` can reference them individually.

Variables

Variable	Type	Notes
WBP_Inventory_Player	WBP_Inventory	Left panel — player's inventory
WBP_Inventory_Loot	WBP_Inventory	Right panel — container's inventory
PlayerInventoryRef	AC_InventoryComponent	Cached for close logic
LootInventoryRef	AC_InventoryComponent	Cached for close logic

Functions

SetupLootPanel(PlayerInventory, LootInventory: AC_InventoryComponent)

1. SET `PlayerInventoryRef` = PlayerInventory
2. SET `LootInventoryRef` = LootInventory
3. GET `WBP_Inventory_Player` → `SetComponentReference(PlayerInventory)`
4. GET `WBP_Inventory_Loot` → `SetComponentReference(LootInventory)`

Each `SetComponentReference` call initialises its widget independently — creates slots sized to that component's `SlotMax`, binds refresh, and populates display.

CloseLootPanel

1. Call `CloseInventory` on `PlayerInventoryRef` — restores input mode and hides cursor
2. `Remove from Parent` on self

Design notes

- Inner WBP_Inventory exit buttons hidden — single close button on WBP_LootPanel
- Loot panel owns its own input mode change on open and close — `OpenInventory`/`CloseInventory` on the component are bypassed
- Drag and drop works cross-inventory — `MoveItem` fires `OnInventoryUpdated` on both components so both panels refresh

---

BP_DragDropOperation

Variable	**Type**
ItemKey	Name
Amount	Integer
SourceInventory	AC_InventoryComponent
SourceSlotIndex	Integer

BP_Pickup_Base ✓

Single item per world pickup. Children override `OnPickup`/`OnUse` for specific behaviour.

Components: StaticMesh, SphereComponent (object type: Items), WidgetComponent (Screen space)

Logic:

- Overlap begin → `GetComponentByClass`, store Inventory, `SetInteractable(self)`, show widget
- Overlap end → `ClearInteractable`, clear refs, hide widget
- Interact → `AddItem` → 0: destroy self, >0: log Warning, -1: log Error

---

BP_Lootable_Base ✓

Container actor with its own `AC_InventoryComponent`. Implements `BPI_Interactable`. On Interact: constructs `WBP_LootPanel`, calls `SetupLootPanel(PlayerInventoryComponent, self.InventoryComponent)`, sets input mode to Game and UI, shows cursor.

Key Design Decisions

- One table — `DT_Items` for all types, enum for categorisation
- Minimum slot state — ItemKey + Quantity only
- Fixed length Slots array — always `SlotMax` entries, empty slots use None key and 0 quantity. Never use `ADD` or `Remove Index` after initialisation
- `Set Array Elem` only — all slot writes use `Set Array Elem` at known indices to preserve array length
- Component owns all state changes — nothing touches Slots array directly
- UI is a mirror — reacts to `OnInventoryUpdated`, never modifies data
- `SlotsUsed` refreshed on use — derived from `Slots.Length`, self-correcting
- Item behaviour in item — effects in child Blueprints, component stays generic
- Single player only
- World pickups are single items — multiples use loot container
- Self contained input — component manages its own IMC and bindings
- Started trigger on `IA_Inventory` — prevents multiple toggles
- `MoveItem` separate from `AddItem` — drag and drop uses direct array manipulation
- `MoveItem` fires `OnInventoryUpdated` on both components — required for cross-inventory panel refresh
- Pre-created slots — all `SlotMax` slots created via `SetComponentReference`, not in widget Construct
- Construct over Create Widget — passes Index and InventoryRef to each slot at creation
- Bind in Construct, init in SetComponentReference — event binding must live in EventGraph (UE5 limitation), slot creation and ref setting live in `SetComponentReference`
- `SetComponentReference` triggers refresh via dispatcher — not called directly, ensures slot widgets are fully ready before `InitSlot` runs
- IsValid guard on Enhanced Input subsystem — prevents "Accessed None" error on PIE stop
- Esc closes all UI(the key can easily be changed, it’s in the AC_Inventory)
